ARM M4 Exception

26 Oct 2022

1. What is Exception?

An exception is **any condition** that requires the core to **halt normal execution** and **execute a dedicated software routine** known as an **exception handler** associated with each **exception type**.

4 Categories of exception:

- Reset
- Supervisor Call (SVCall)
- Fault
- Interrupt

Exception properties:

- An exception number.
- A priority level.
- A vector in memory

Exception States:

- Inactive
- Pending
- Active
- Active and pending

synchronous exception
asynchronous exception

2. Exception model

1. Exception states
2. Exception types
3. Exception handlers
4. Vector table
5. Exception priorities
6. Interrupt priority grouping
7. Exception entry and return

3. Exception states

No.	States	Descriptions
1	Inactive	The exception is **not active and not pending**.
2	**Pending**	The exception is **waiting** to be serviced by the processor. An interrupt request from a peripheral or from software can change the state of the corresponding interrupt to pending.
3	**Active**	An exception that is **being serviced** by the processor but has not completed.
4	**Active and pending**	The exception is being serviced by the processor and there is a pending exception from the **same source**.

4. Exception types

No.	Exception types	Descriptions
1	**Reset**	Reset is invoked on power up or a warm reset. The exception model treats reset as a special form of exception. When reset is asserted, the operation of the processor stops, potentially at any point in an instruction. When reset is deasserted, execution restarts from the address provided by the reset entry in the vector table. Execution restarts as privileged execution in Thread mode.
2	**NMI**	A Non Maskable Interrupt (NMI) can be signaled by a peripheral or triggered by software. This is the highest priority exception other than reset. It is permanently enabled and has a fixed priority of -2. NMIs cannot be: • masked or prevented from activation by any other exception • preempted by any exception other than Reset.
3	**HardFault**	A HardFault is an exception that occurs because of an error during exception processing, or because an exception cannot be managed by any other exception mechanism. HardFaults have a fixed priority of -1, meaning they have higher priority than any exception with configurable priority.

4. Exception types

No.	Exception types	Descriptions
4	**MemManage**	A MemManage fault is an exception that occurs because of a memory protection related fault. The the fixed memory protection constraints determines this fault, for both instruction and data memory transactions. This fault is always used to abort instruction accesses to *Execute Never* (XN) memory regions.
5	**BusFault**	A BusFault is an exception that occurs because of a memory related fault for an instruction or data memory transaction. This might be from an error detected on a bus in the memory system.
6	**UsageFault**	A UsageFault is an exception that occurs because of a fault related to instruction execution. This includes: • an undefined instruction • an illegal unaligned access • invalid state on instruction execution • an error on exception return. The following can cause a UsageFault when the core is configured to report them: • an unaligned address on word and halfword memory access • division by zero.

4. Exception types

No.	Exception types	Descriptions
7	**SVCall**	A *supervisor call* (SVC) is an exception that is triggered by the SVC instruction. In an OS environment, applications can use SVC instructions to access OS kernel functions and device drivers.
8	**PendSV**	PendSV is an interrupt-driven request for system-level service. In an OS environment, use PendSV for context switching when no other exception is active.
9	**SysTick**	A SysTick exception is an exception the system timer generates when it reaches zero. Software can also generate a SysTick exception. In an OS environment, the processor can use this exception as system tick.
10	**Interrupt (IRQ)**	A interrupt, or IRQ, is an exception signalled by a peripheral, or generated by a software request. All interrupts are asynchronous to instruction execution. In the system, peripherals use interrupts to communicate with the processor.

5. Exception handlers

No.	Handler	Descriptions
1	**Interrupt Service Routines (ISRs)**	The IRQ interrupts are the exceptions handled by ISRs.
2	**Fault handlers**	HardFault, MemManage fault, UsageFault, and BusFault are fault exceptions handled by the fault handlers.
3	**System handlers**	NMI, PendSV, SVCall SysTick, and the fault exceptions are all system exceptions that are handled by system handlers.

6. Vector table

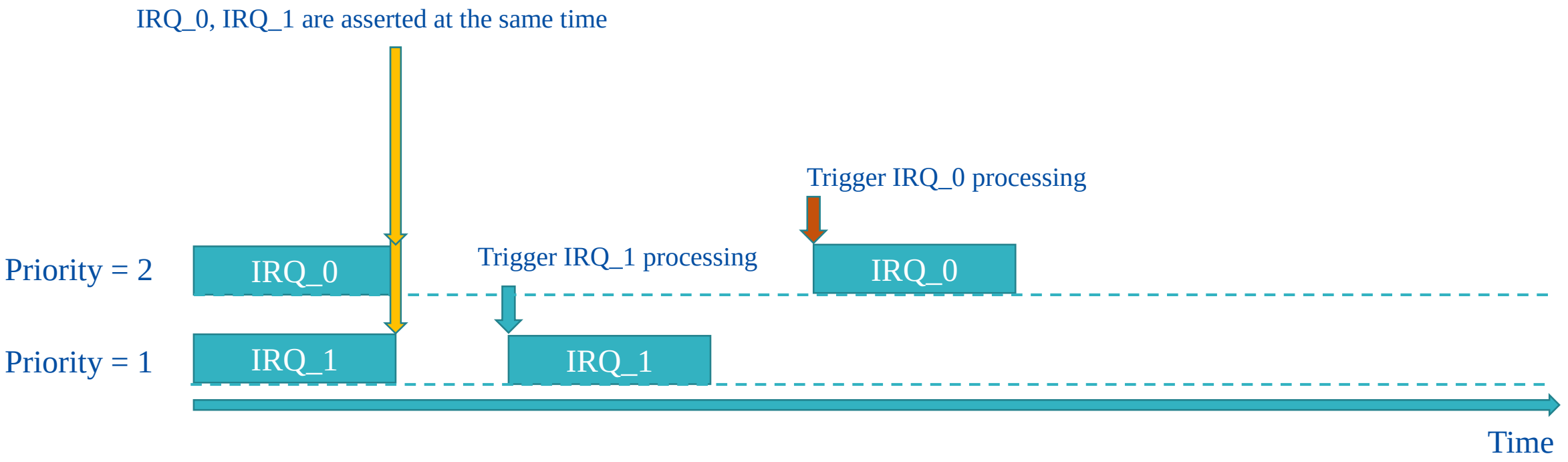
Exception Num	IRQ Number	Exception type	Priority	Vector address or offset	Activation
0	-	Initial SP value	-	0x00000000	-
1	-	Reset	-3, the highest	0x00000004	Asynchronous
2	-14	NMI	-2	0x00000008	Asynchronous
3	-13	HardFault	-1	0x0000000C	-
4	-12	MemManage	Configurable	0x00000010	Synchronous
5	-11	BusFault	Configurable	0x00000014	Synchronous when precise, asynchronous when imprecise
6	-10	UsageFault	Configurable	0x00000018	Synchronous
7 - 10	-	Reserved	-	-	-
11	-5	SVCall	Configurable	0x0000002C	Synchronous
12 - 13	-	Reserved	-	-	-
14	-2	PendSV	Configurable	0x00000038	Asynchronous
15	-1	SysTick	Configurable	0x0000003C	Asynchronous
16	0	Interrupt (IRQ0)	Configurable	0x00000040	Asynchronous
17	1	Interrupt (IRQ…)	Configurable	0x0000000…	Asynchronous
18	2	Interrupt (IRQn)	Configurable	0x0000000…	Asynchronous

7. Exception priorities

All exceptions have an associated priority, with:

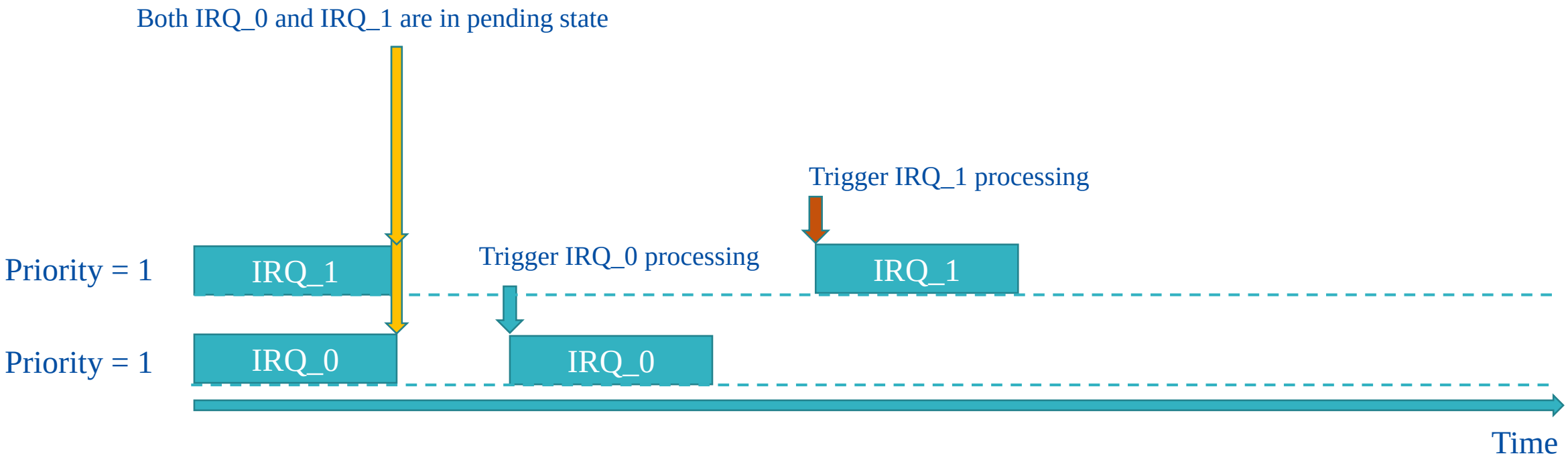
- A **lower** priority value indicating a **higher** priority.
- Configurable priorities for all exceptions **except** Reset, HardFault, and NMI.
- If software does **not configure** any priorities, then all exceptions with a configurable priority have a priority of **ZERO**.
- **Reset, HardFault, and NMI** exceptions, with **fixed negative priority** values, always have higher priority than any other exception.

7. Exception priorities



7. Exception priorities

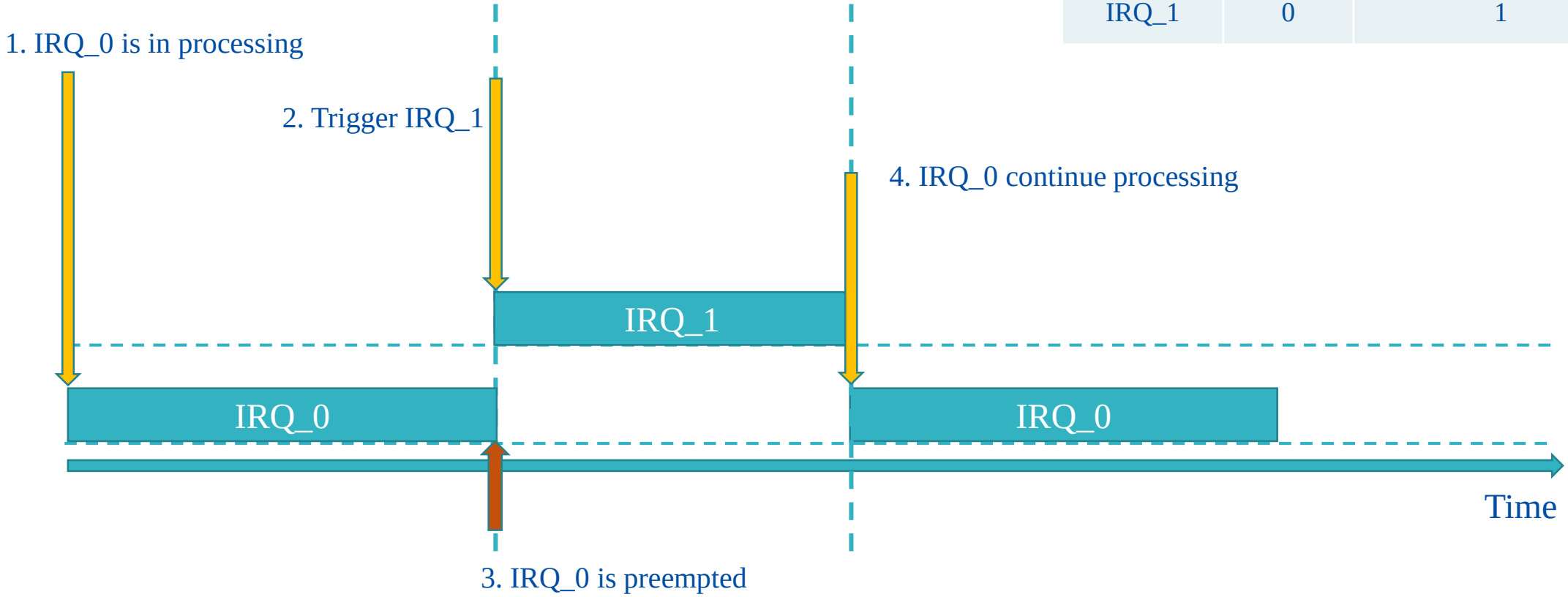
If multiple pending exceptions have the **same** priority, the pending exception with the **lowest exception number** takes precedence



7. Exception priorities

When the processor is executing an exception handler, the exception handler is preempted if a higher priority exception occurs

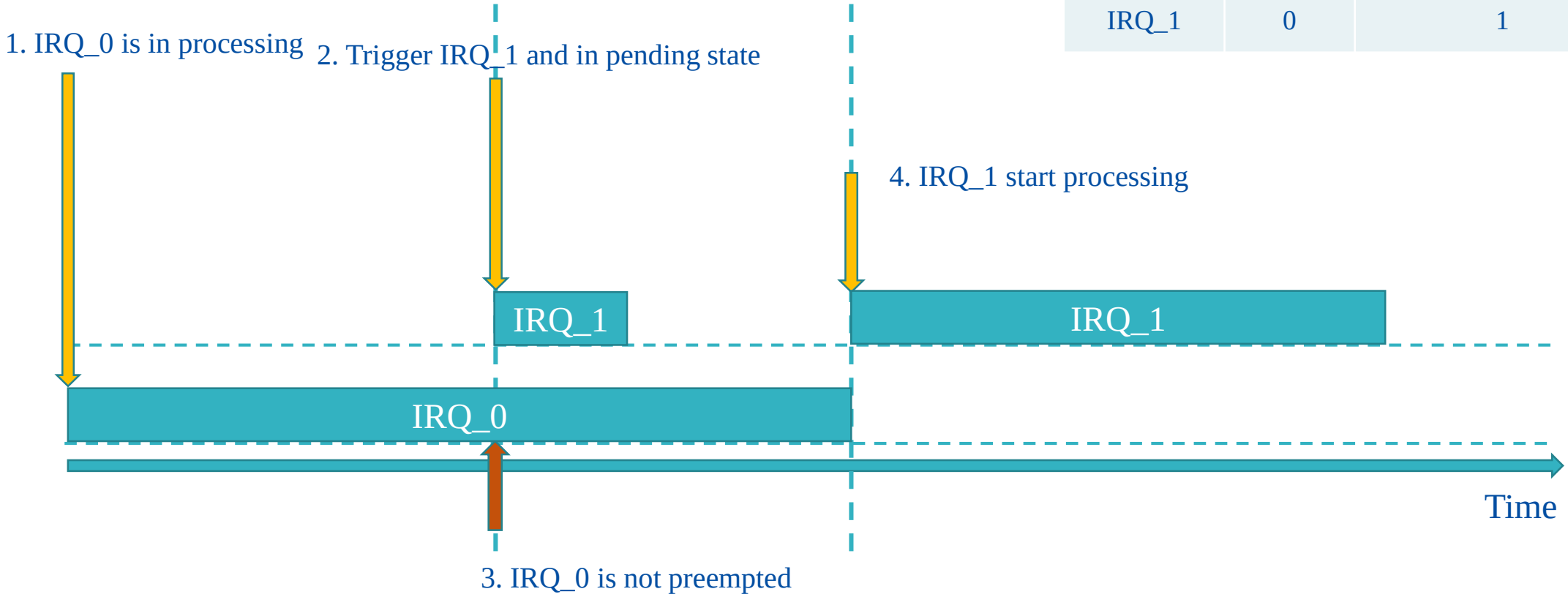
Exception	Priority	Exception number
IRQ_0	1	0
IRQ_1	0	1



7. Exception priorities

If an exception occurs with the same priority as the exception being handled, the handler is not preempted, irrespective of the exception number. However, the status of the new interrupt changes to **pending**.

Exception	Priority	Exception number
IRQ_0	0	0
IRQ_1	0	1



8. Interrupt priority grouping

To increase priority control in systems with interrupts, the NVIC supports priority grouping. This divides each interrupt priority register entry into two fields:

- An upper field that defines the group priority
- A lower field that defines a subpriority within the group.

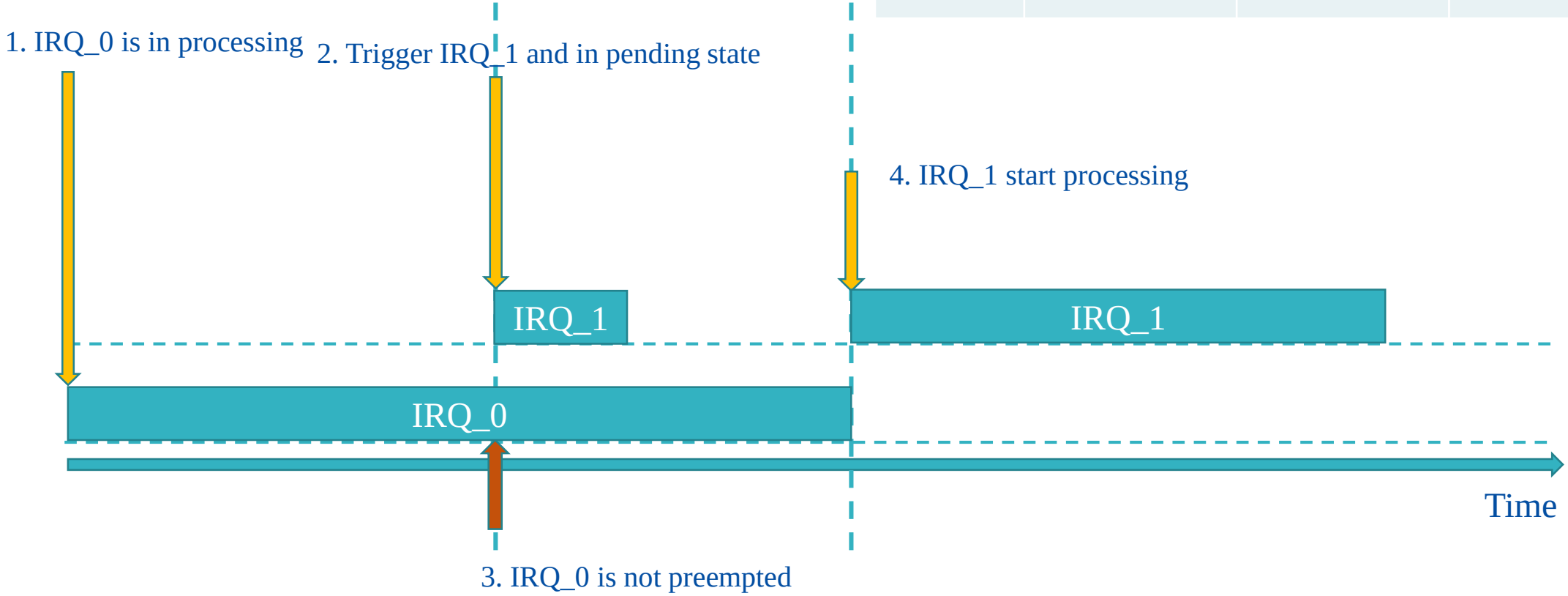


Interrupt priority register

8. Interrupt priority grouping

When the processor is executing an interrupt exception handler, another interrupt with the **same group priority** as the interrupt being handled does **not preempt** the handler

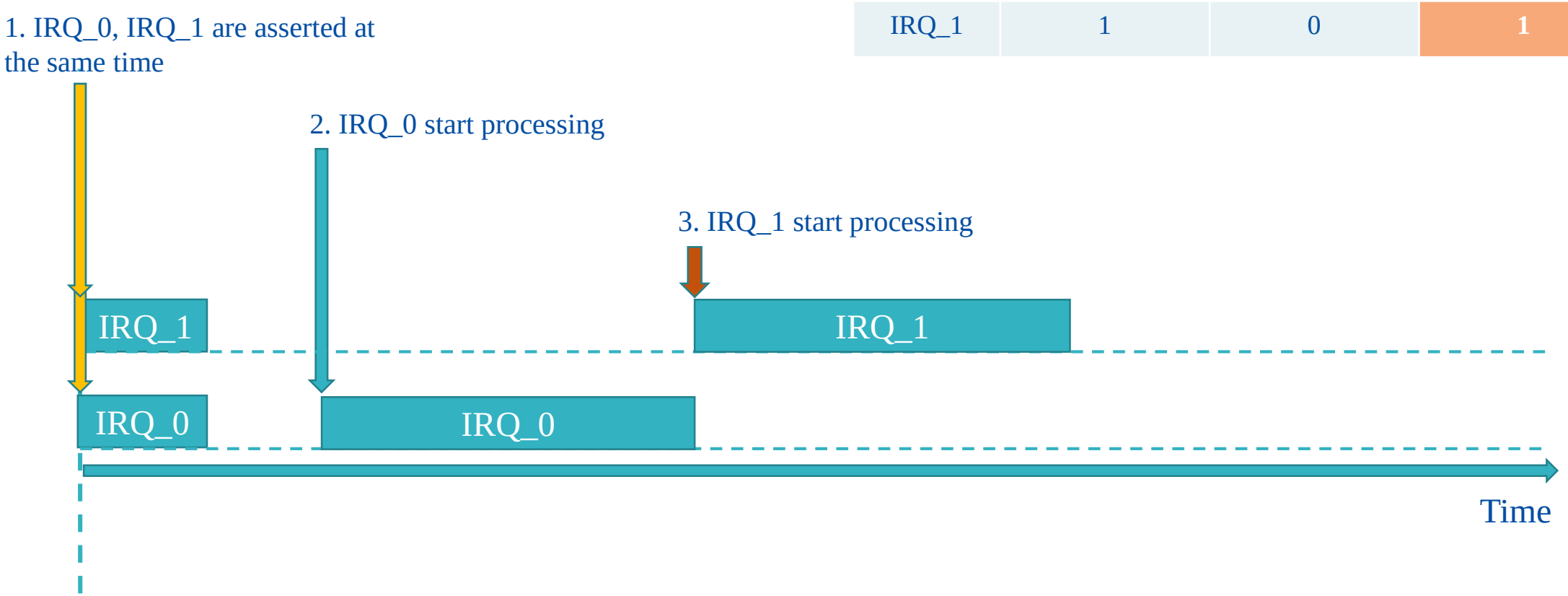
Exception	Exception Num	Group Priority	Sub Priority
IRQ_0	0	0	1
IRQ_1	1	0	0



8. Interrupt priority grouping

If multiple pending interrupts have the **same group** priority, the **subpriority** field determines the order in which they are processed

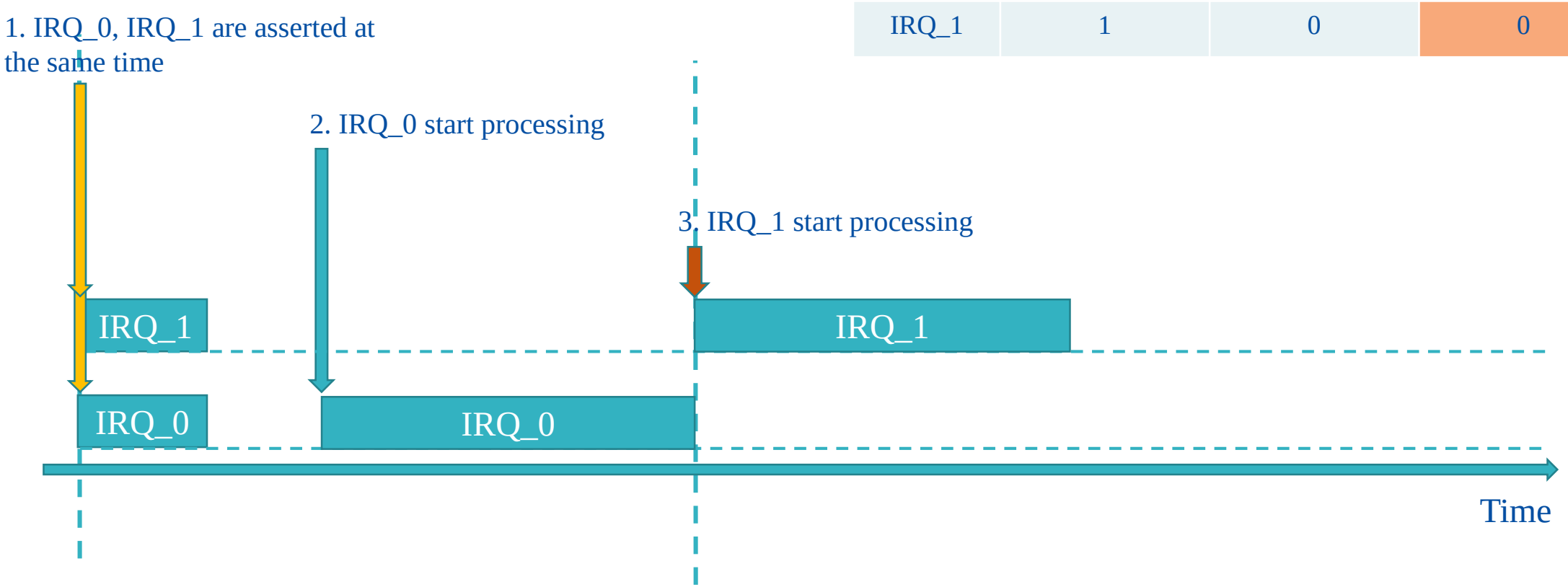
Exception	Exception Num	Group Priority	Sub Priority
IRQ_0	0	0	0
IRQ_1	1	0	1



8. Interrupt priority grouping

If multiple pending interrupts have the **same group** priority and **subpriority**, the interrupt with the lowest IRQ number is processed first.

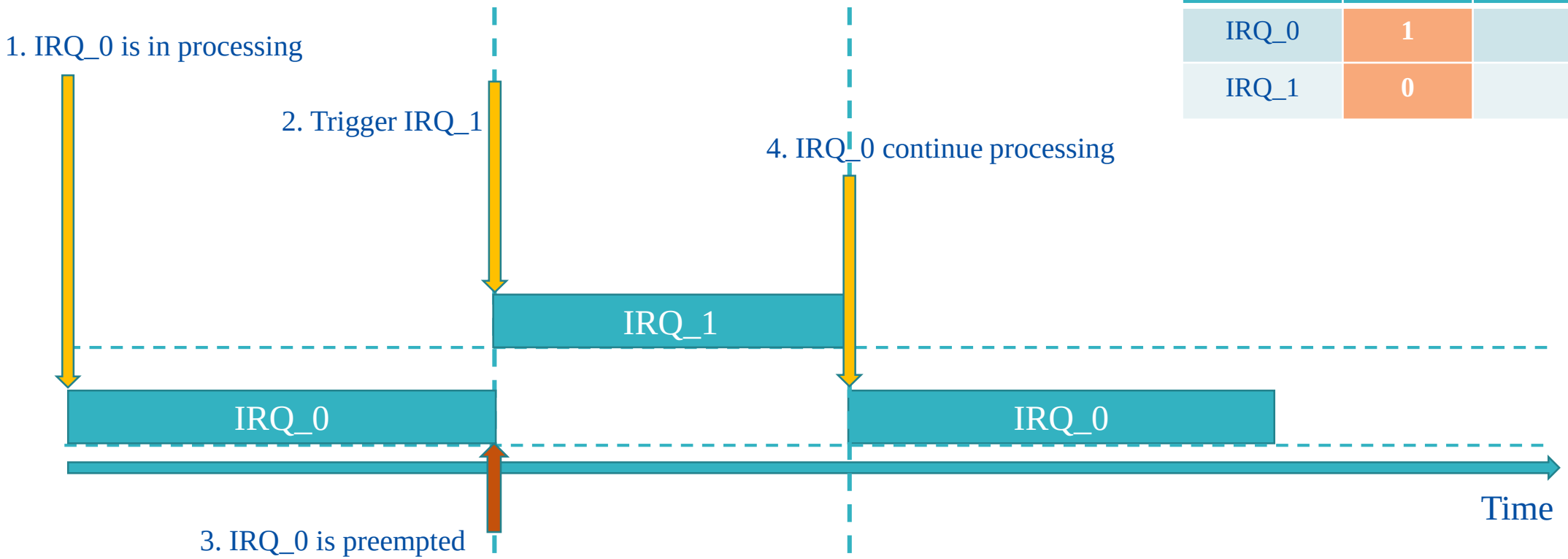
Exception	Exception Num	Group Priority	Sub Priority
IRQ_0	0	0	0
IRQ_1	1	0	0



9. Exception entry and return

No.	Terms	Description
1	**Preemption**	When the processor is executing an exception handler, an exception can preempt the exception handler if its priority is higher than the priority of the exception being handled. When one exception preempts another, the exceptions are called nested exceptions.

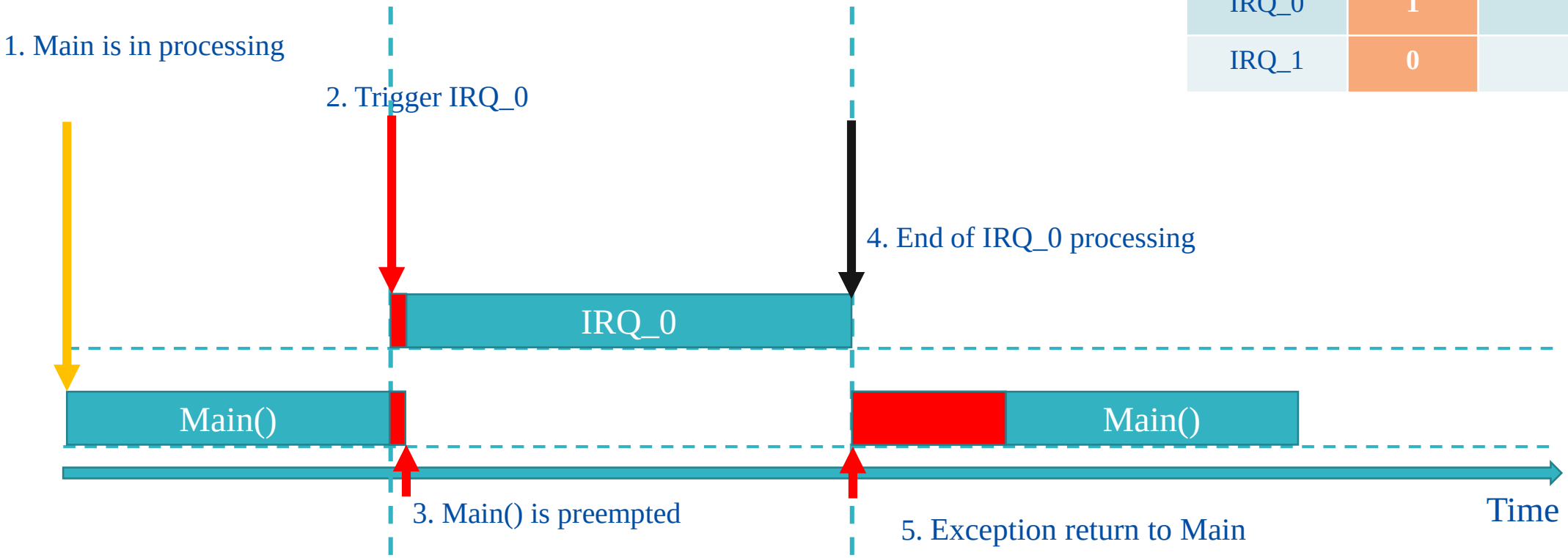
Exception	Priority	Exception number
IRQ_0	1	0
IRQ_1	0	1



9. Exception entry and return

No.	Terms	Description
2	**Return**	This occurs when the exception handler is completed, and: ▪ There is no pending exception with sufficient priority to be serviced ▪ The completed exception handler was not handling a late-arriving exception. The processor pops the stack and restores the processor state to the state it had before the interrupt occurred.

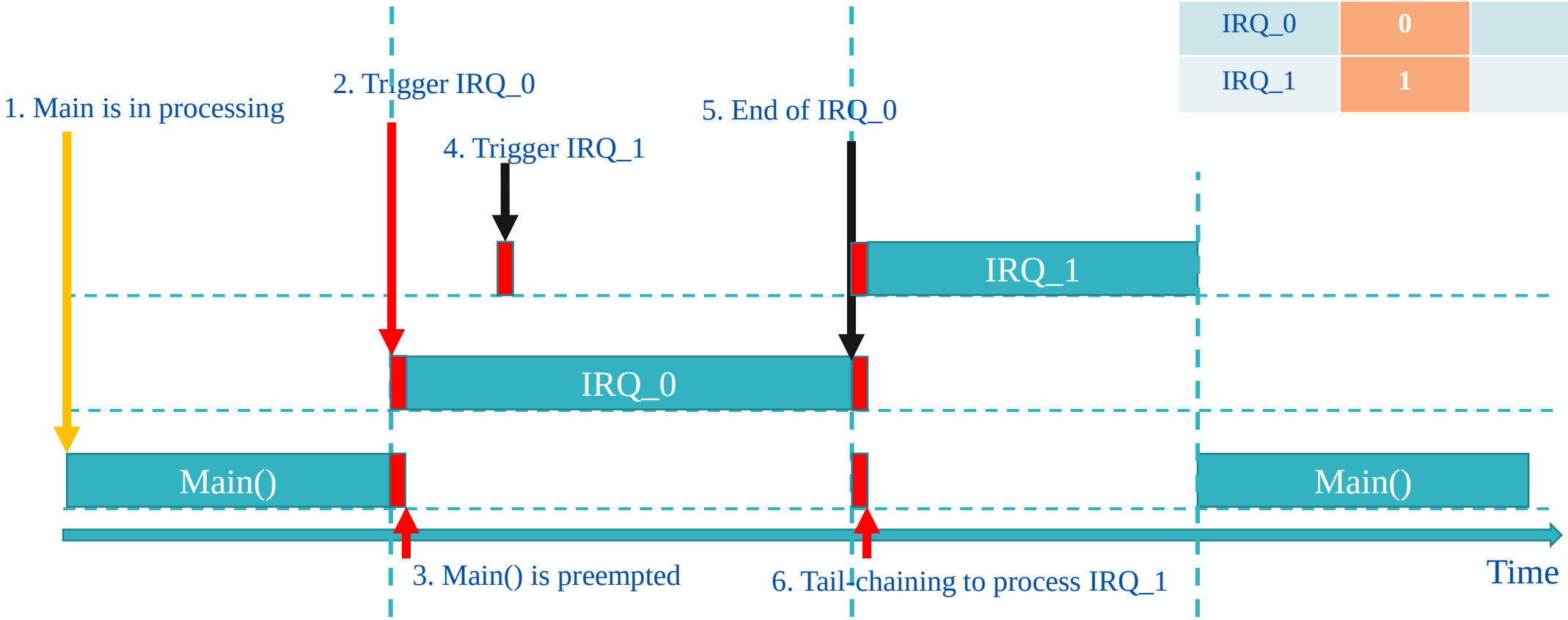
Exception	Priority	Exception number
IRQ_0	1	0
IRQ_1	0	1



9. Exception entry and return

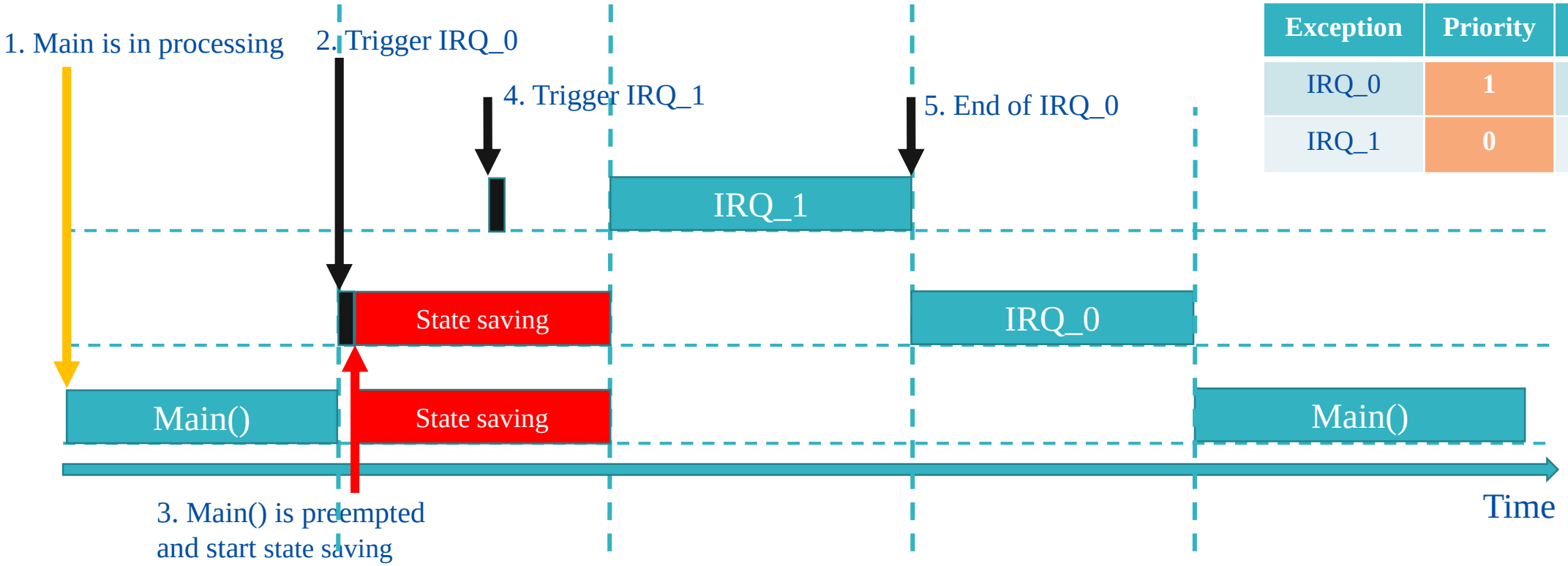
No.	Terms	Description
3	**Tail-chaining**	This mechanism **speeds up** exception servicing. On completion of an exception handler, if there is a pending exception that meets the requirements for exception entry, the stack pop is skipped and control transfers to the new exception handler.

Exception	Priority	Exception number
IRQ_0	0	0
IRQ_1	1	1



9. Exception entry and return

No.	Terms	Description
4	**Late-arriving**	This mechanism **speeds up** preemption. If a higher priority exception occurs during state saving for a previous exception, the processor switches to handle the higher priority exception and initiates the vector fetch for that exception. State saving is not affected by late arrival because the state saved is the same for both exceptions. Therefore the state saving continues uninterrupted. The processor can accept a late arriving exception until the first instruction of the exception handler of the original exception enters the execute stage of the processor. On return from the exception handler of the late-arriving exception, the normal tail-chaining rules apply.

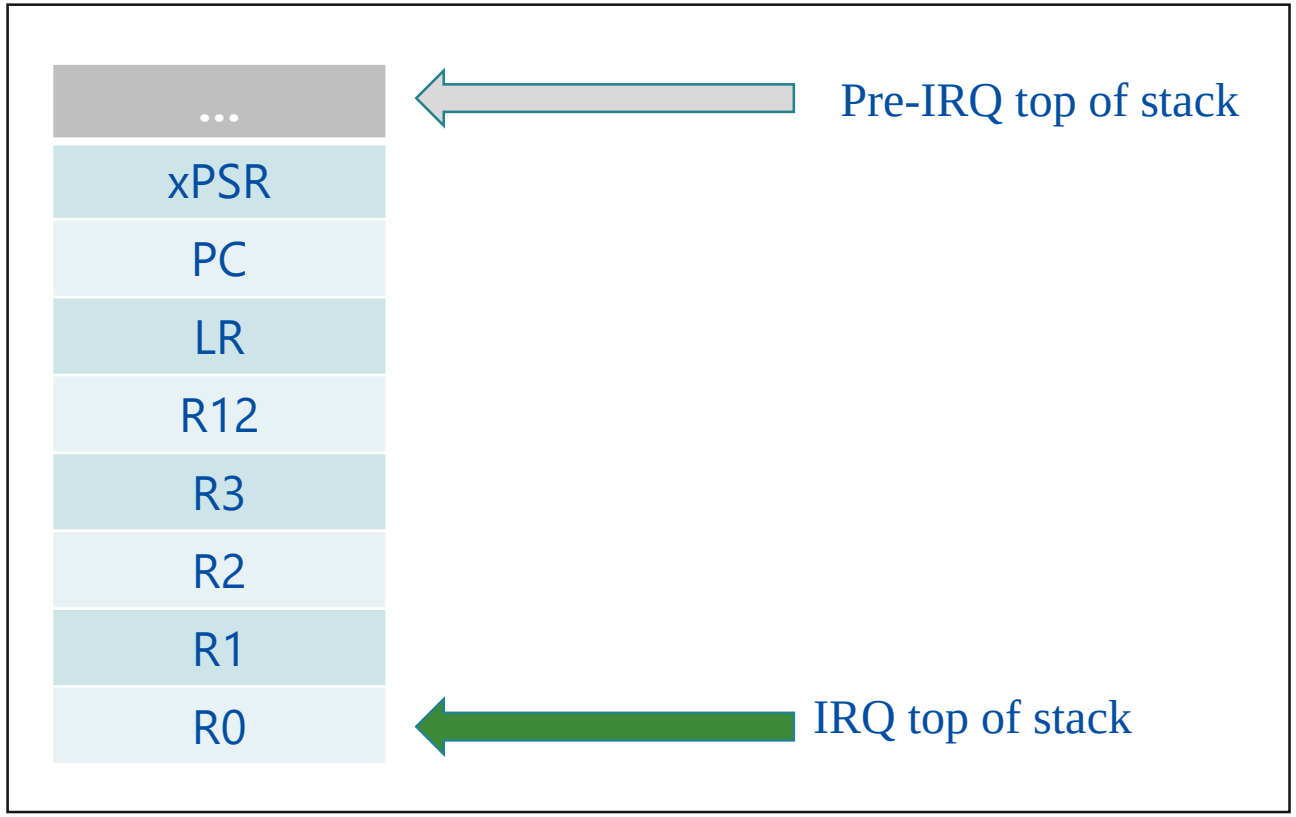


Exception	Priority	Exception number
IRQ_0	1	0
IRQ_1	0	1

9. Exception entry and return

When **Exception Entry** Occur?

1. If the processor is in Thread mode and there is a pending exception
2. If there is an exception being handled and has a new exception with higher priority in pending state



Stack frame without Floating point storage

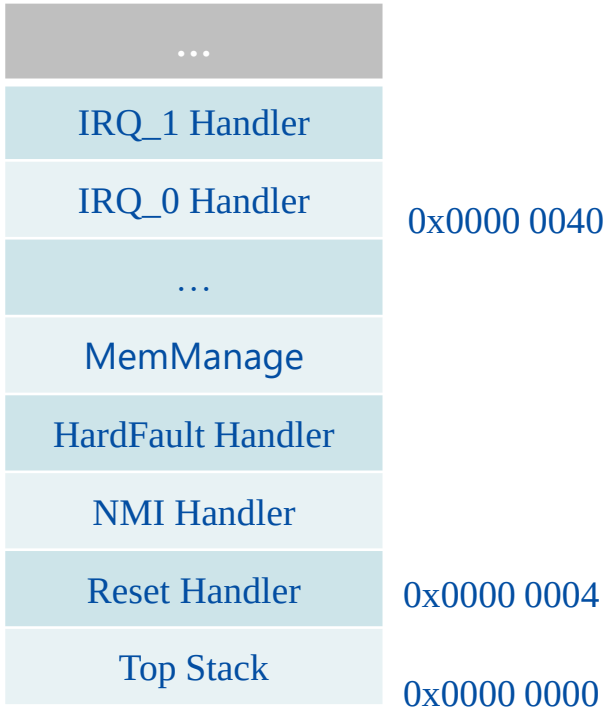
9. Exception entry and return

How **Exception entry** operation?

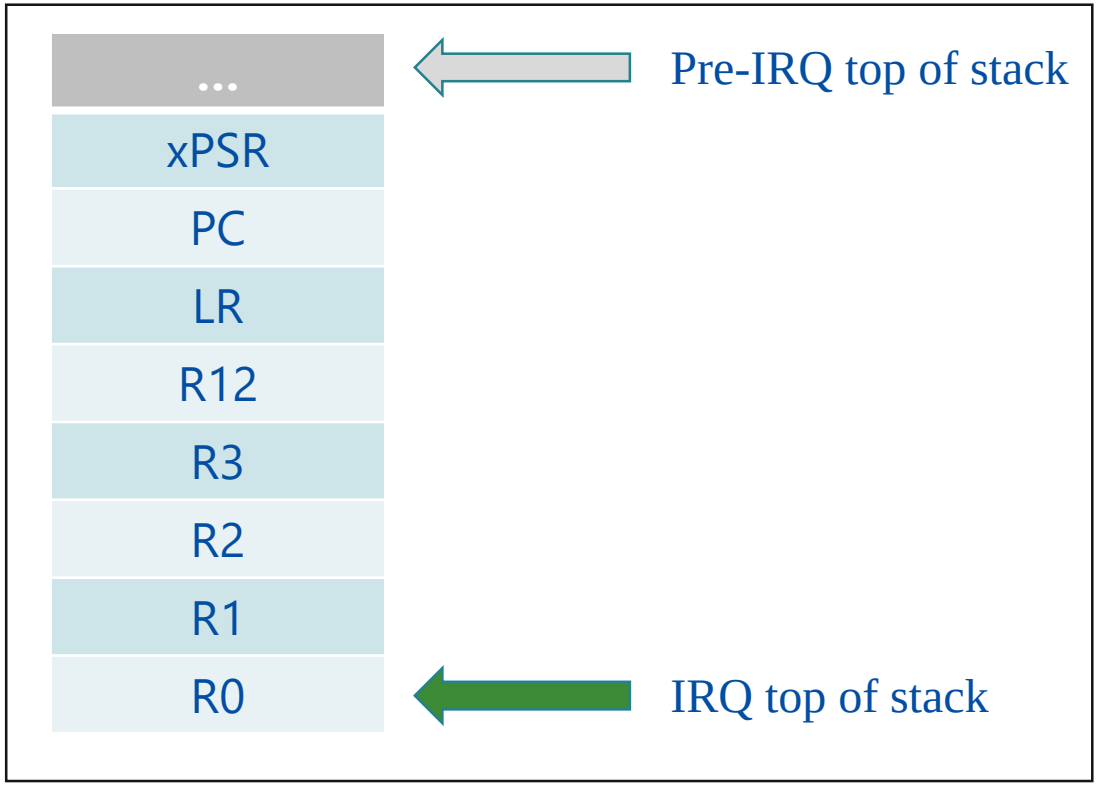
Perform stacking operation

Perform a vector fetch reads the exception handler start address from the vector table

Note: The processor writes an EXC_RETURN value to the LR when perform stacking



Vector Table



Stack frame without Floating point storage

9. Exception entry and return

When **Exception Return** Occur?

Exception return occurs when the processor is in **Handler mode** and executes one of the following instructions to load the EXC_RETURN value into the PC:

- An **LDM** or **POP** instruction that loads the PC
- An **LDR** instruction with PC as the destination
- A **BX** instruction using any register.

What is **EXC_RETURN**?

EXC_RETURN is the value loaded into the LR on exception entry

This indicates:

- Stack pointer corresponds to the stack frame
- Operation mode the processor was in before the entry occurred.

9. Exception entry and return

What is **EXC_RETURN**?

EXC_RETURN is the value loaded into the LR on exception entry

This indicates:

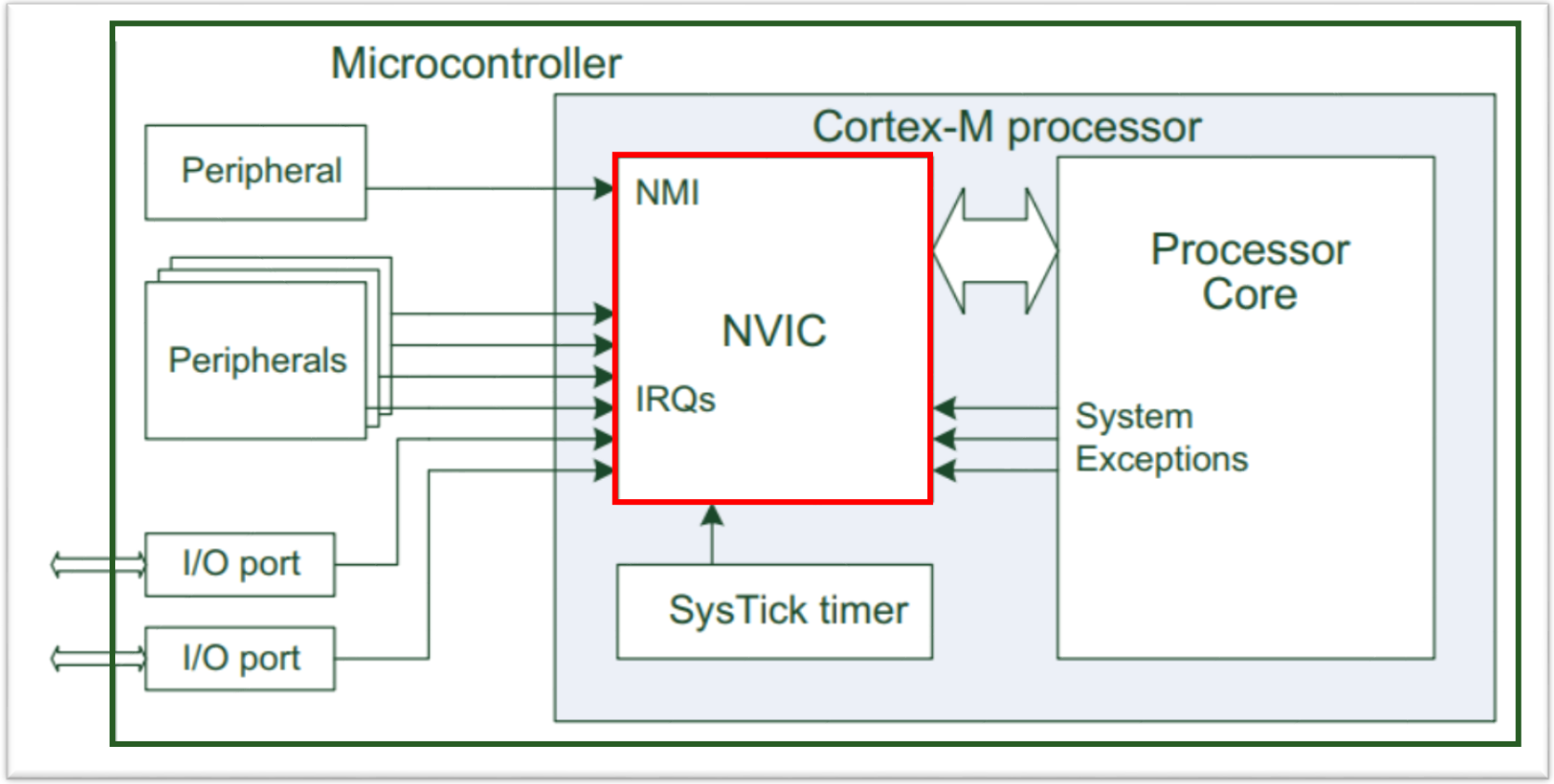
- ❑ Stack pointer corresponds to the stack frame
- ❑ Operation mode the processor was in before the entry occurred.



EXC_RETURN[31:0]	Description
0xFFFFFFF1	Return to Handler mode, exception return uses **non**-floating-point state from the MSP and execution uses MSP after return.
0xFFFFFFF9	Return to Thread mode, exception return uses **non**-floating-point state from MSP and execution uses MSP after return.
0xFFFFFFFD	Return to Thread mode, exception return uses **non**-floating-point state from the PSP and execution uses PSP after return.
0xFFFFFFE1	Return to Handler mode, exception return uses floating-point state from MSP and execution uses MSP after return.
0xFFFFFFE9	Return to Thread mode, exception return uses floating-point state from MSP and execution uses MSP after return.
0xFFFFFFED	Return to Thread mode, exception return uses floating-point state from PSP and execution uses PSP after return.

Cortex-M4 Peripherals - NVIC

NVIC stands for **N**ested **V**ectored **I**nterrupt **C**ontroller



Cortex-M4 Peripherals - NVIC

NVIC stands for **N**ested **V**ectored **I**nterrupt **C**ontroller

No.	Supported Feature
1	An implementation-defined number of interrupts, in the range **1-240** interrupts
2	A programmable priority level of 0-255
3	Level and pulse detection of interrupt signals.
4	Dynamic reprioritization of interrupts.
5	Grouping of priority
6	Interrupt tail-chaining.
7	An external Non Maskable Interrupt (NMI)
8	Optional WIC, providing ultra-low power sleep mode support.


Thank you